Robotics: Computational Motion Planning — Mini Project Report

Search-Based Planning vs. Reinforcement Learning for a Kinodynamically Constrained Agent in Super Mario Bros

Aditya Khairnar — C221

1. Introduction

Super Mario Bros is treated here as a **kinodynamic motion planning problem** rather than as a game. Mario's configuration is q = (x, y, vx, vy, on_ground, powerup): position **and velocity**, because two agents at the same position with different velocities have entirely different reachable sets. Gravity, momentum and near-zero mid-air control authority are genuine differential constraints — a jump is a ballistic commitment that must be correct before take-off — and the enemies are moving obstacles.

Two solvers are compared on the same task under one harness. The first is **A* search over motion primitives**: short fixed button sequences form the graph edges, so every edge is dynamically feasible by construction; edge cost is duration in frames, and the heuristic h(s) = (xgoal − x) / 3 px per frame is admissible because 3 px/frame is a hard engine cap, and consistent because it is 1-Lipschitz in x. Expanding an edge restores an emulator snapshot, runs the primitive and snapshots again, so the emulator is both the transition function and the collision checker and there is zero modelling error. The second solver is **PPO trained from pixels** (84×84 grayscale, 4 stacked frames, NatureCNN): 5,000,672 agent steps — 20.0 M emulated frames — in 74.1 minutes on 8 parallel workers.

Because the emulator is fully deterministic, an executed plan would otherwise score 100% trivially, which would say something about the emulator rather than about planning. Every agent is therefore also evaluated under **sticky actions at p = 0.25** (Machado et al., 2018), applied at the frame level: the previously executed control may repeat instead of the commanded one. This models **actuation uncertainty** — the planner's model stays exact, but the executed control is not the commanded one. The central question follows directly: when execution is unreliable, does a committed plan or a learned policy hold up better, and what does each one cost? To separate the **search** from the **feedback**, the same plan is run in two agents: open-loop, which executes the plan it computed at the start, and closed-loop, which replans a 160 px receding horizon from the observed state every 15 frames.

2. Results

All four agents run through one harness on level 1-1 — same env construction, same stepping loop, same metrics — with n = 3 deterministic episodes (repeats verify reproducibility rather than estimate a rate) and n = 20 perturbed episodes.

Agent	Deterministic	Sticky p = 0.25	Decision latency	Frames to goal[a]
Random baseline	0.00	0.00	~0	—
A* open-loop	**1.00**	**0.15**	~0 (replay)	1054
A* closed-loop	**1.00**	**0.80**	28.6 ms	1100
PPO (pixels)	1.00	0.35	0.7 ms	1091

Three readings follow. First, **open-loop planning collapses under perturbation, 1.00 → 0.15**: a plan is a memorised control sequence with no feedback, so once execution drifts nothing in the agent notices. Second, **closed-loop replanning recovers it, 0.15 → 0.80** — same planner, same primitives, same heuristic, the only difference being that the agent re-solves from where it actually is. Third, **closed-loop A* beats PPO by more than 2× under the same noise** (0.80 against 0.35), despite PPO having consumed 279× more emulated frames. A learned policy is not inherently more robust to actuation noise than an explicit replanner.

The failure modes are as informative as the rates. Of open-loop's 17 failed episodes, **6 are falls into pits** — six times as often as any other agent, which fall in at most once. A pit is exactly where committed, well-timed ballistic motion matters: a jump launched a frame or two late cannot be corrected mid-air, and there is no ground left to recover on. Pit deaths are the signature of open-loop timing drift. Robustness also costs a little optimality: open-loop's successful runs take 1054 frames against the planner's 1040-frame admissible bound, while closed-loop takes 1100, about 4% slower, because it repeatedly commits to a fresh horizon instead of following one globally optimal route.

Cost is paid at different times. PPO front-loads 20.0 M emulated frames into training and then decides in 0.7 ms; the planner spends 71,637 frames searching and pays 28.6 ms at every decision — a 40× run-time gap against a 279× training-time gap. On level 1-2, which neither agent prepared for, nobody finishes, but **closed-loop A* reaches x = 950 with the algorithm unchanged while PPO stops at x = 354**, barely past the random baseline's 355. Searching the dynamics transfers; a fitted policy does not. One engineering result follows from consistency: no expanded node is ever reopened, so the 272 KB snapshots are kept only for the frontier — a measured peak of 239 MB rather than roughly 1.1 GB.

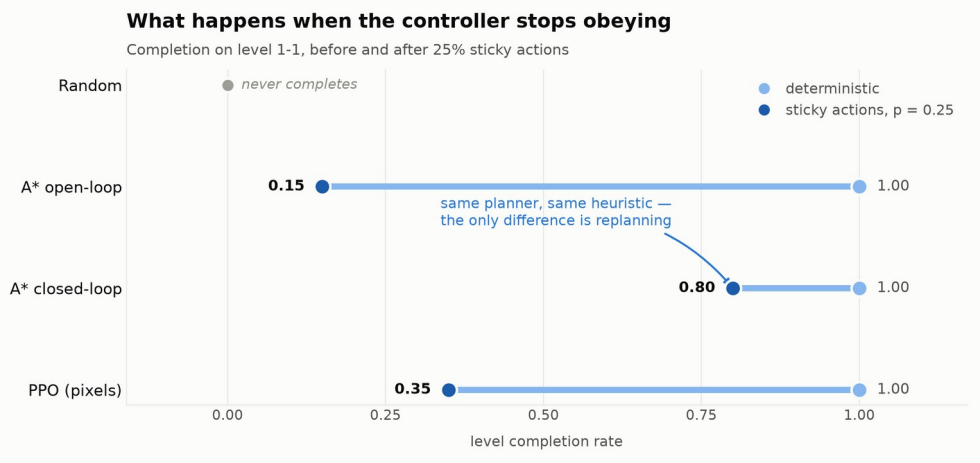


Figure 1 — completion on level 1-1 before (light) and after (dark) 25% sticky actions; the length of each line is what that approach loses to execution noise. [a] Mean frames to the flag over completed episodes in the perturbed condition, read from phase5_summary_main.csv.

3. Conclusion

The project's finding is that **feedback, not the search, is what makes planning robust**. An identical A* planner moves from 0.15 to 0.80 completion under identical noise purely by replanning from the observed state, and that single change also outperforms a policy trained on 279× more experience. The honest reading is a trade rather than a winner: a model, privileged state and 28.6 ms of search per decision, against pixels, 20 M samples and 0.7 ms of inference. The limitations are real and stated — the planner is resolution-complete rather than complete; it does not solve level 1-2, because an x-distance heuristic carries no information about vertical structure and 1-2's exit is not a flagpole; A* reads privileged RAM state while PPO reads only pixels, so the agents are not equally endowed; and all results are single-seed. The natural next steps are a heuristic that reasons about vertical structure, plan-guided initialisation of the learner, and a sweep over the sticky probability to find where the two approaches break even.

References

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. IEEE Trans. SSC, 4(2), 100–107.

LaValle, S. M. (2006). Planning Algorithms. Cambridge University Press.

Likhachev, M., Gordon, G., & Thrun, S. (2003). ARA*: Anytime A* with Provable Bounds on Sub-Optimality. NeurIPS 2003.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Machado, M. C., et al. (2018). Revisiting the Arcade Learning Environment. JAIR, 61, 523–562.